



SHARED QUEUES AND MESSAGE PASSING IN OPENMP

Slides originally by Peter Pacheco
Edited substantially by Libby Shoop
for COMP 445, Macalester College

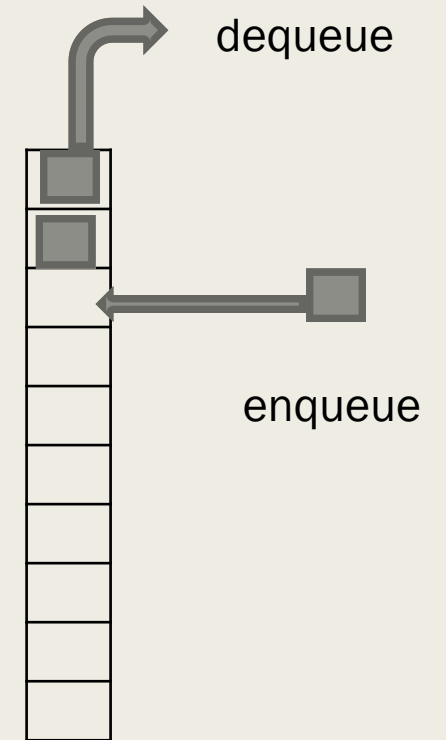


Queues

- Can be viewed as an abstraction of a line of customers waiting to pay for their groceries in a supermarket.
- A natural data structure to use in many multithreaded applications.
- For example, suppose we have several “producer” threads and several “consumer” threads.
 - *Producer threads might “produce” requests for data.*
 - *Consumer threads might “consume” the request by finding or generating the requested data.*

enqueue

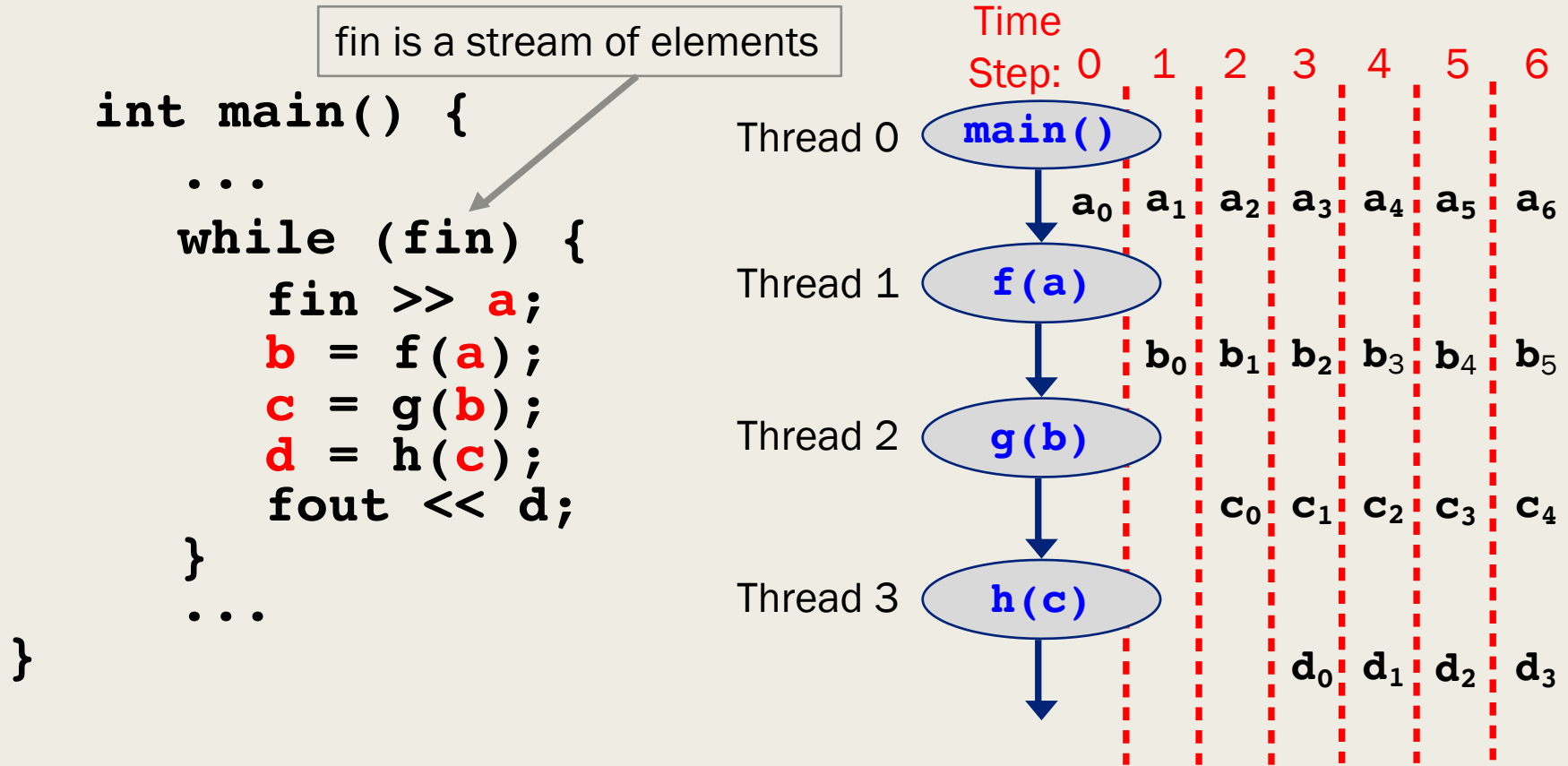
dequeue



Message Passing

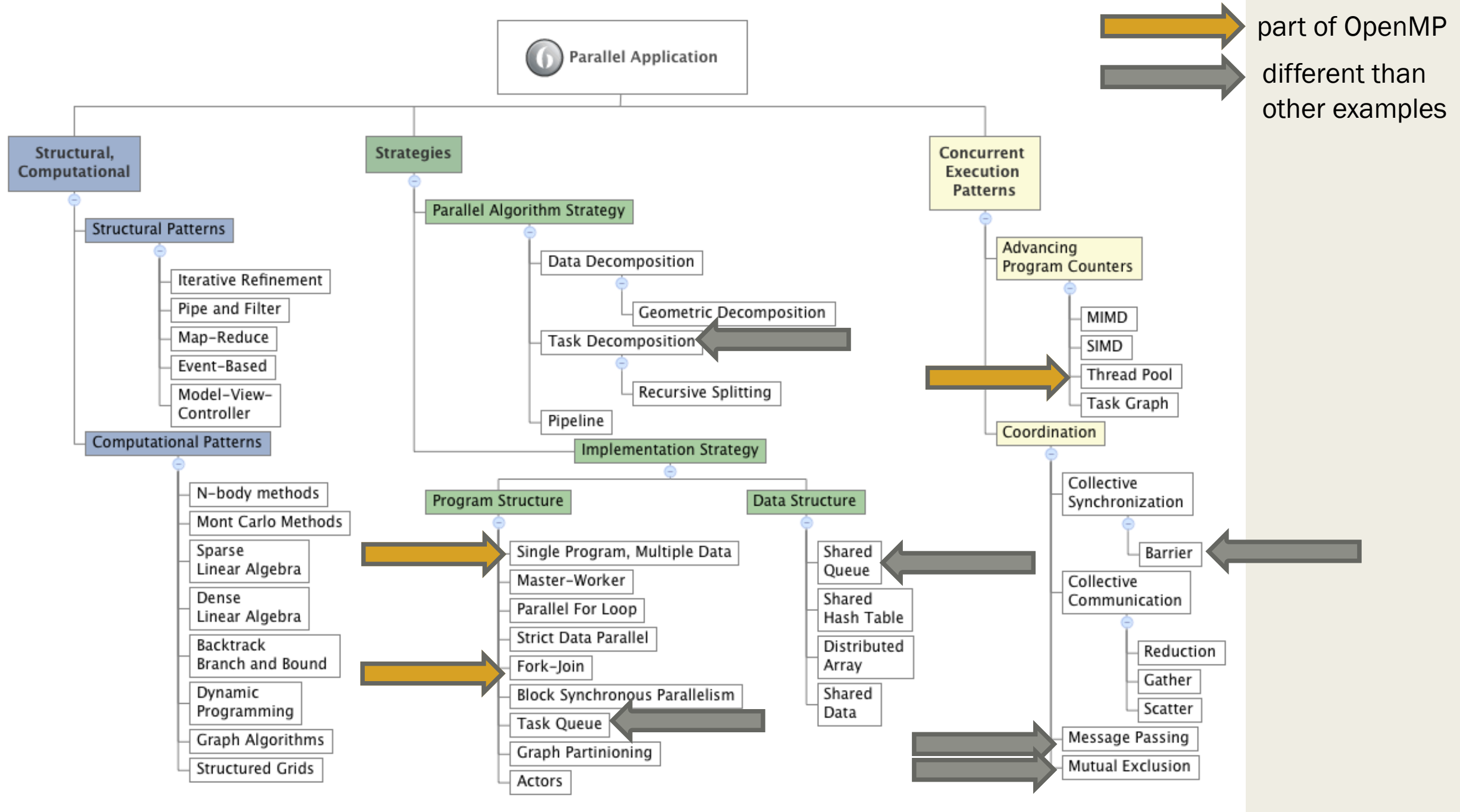
- **Messages** are a general term for data objects of some type that one thread would like another thread to work on.
- **Passing** is the act of ‘sending’ that data object
 - *implementation* of this can have many forms
 - *we’ll examine in parallel shared memory with OpenMP*
- The pattern of message passing can be used as a way to implement various applications
 - *including producer-consumer*
 - *or the pipeline strategy as a special case of that*

Algorithmic Strategy: Pipeline



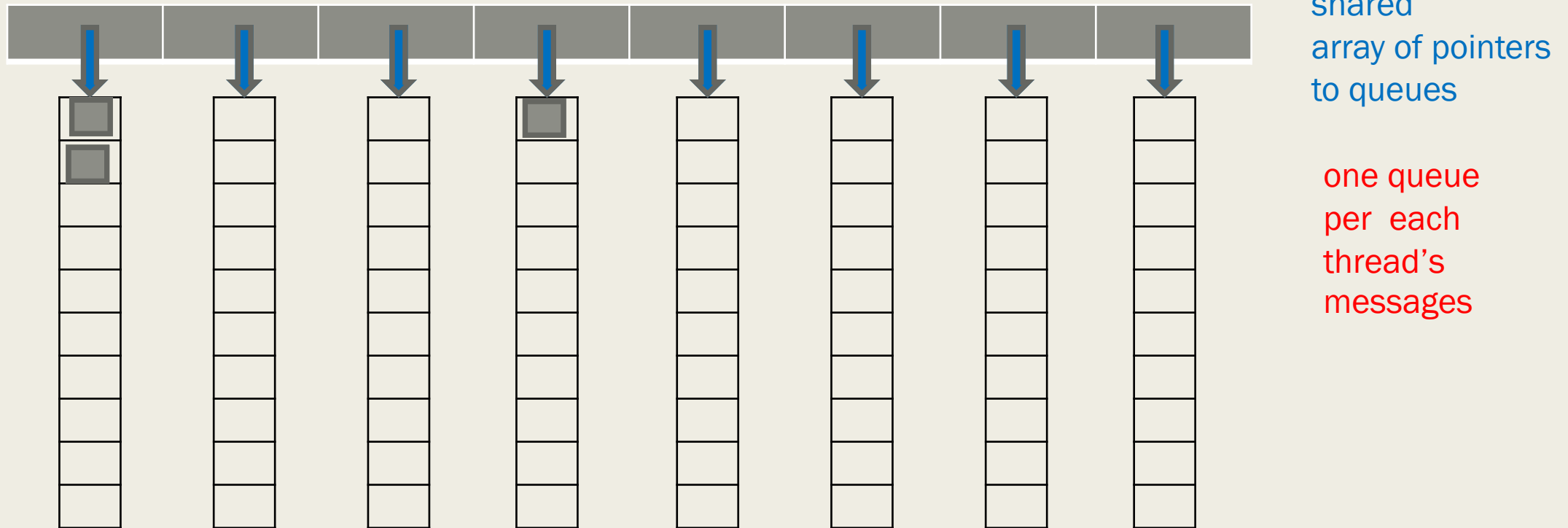
Generic Producer-Consumer

- any thread produces for any other thread
- The code example will have the following patterns



Message-Passing

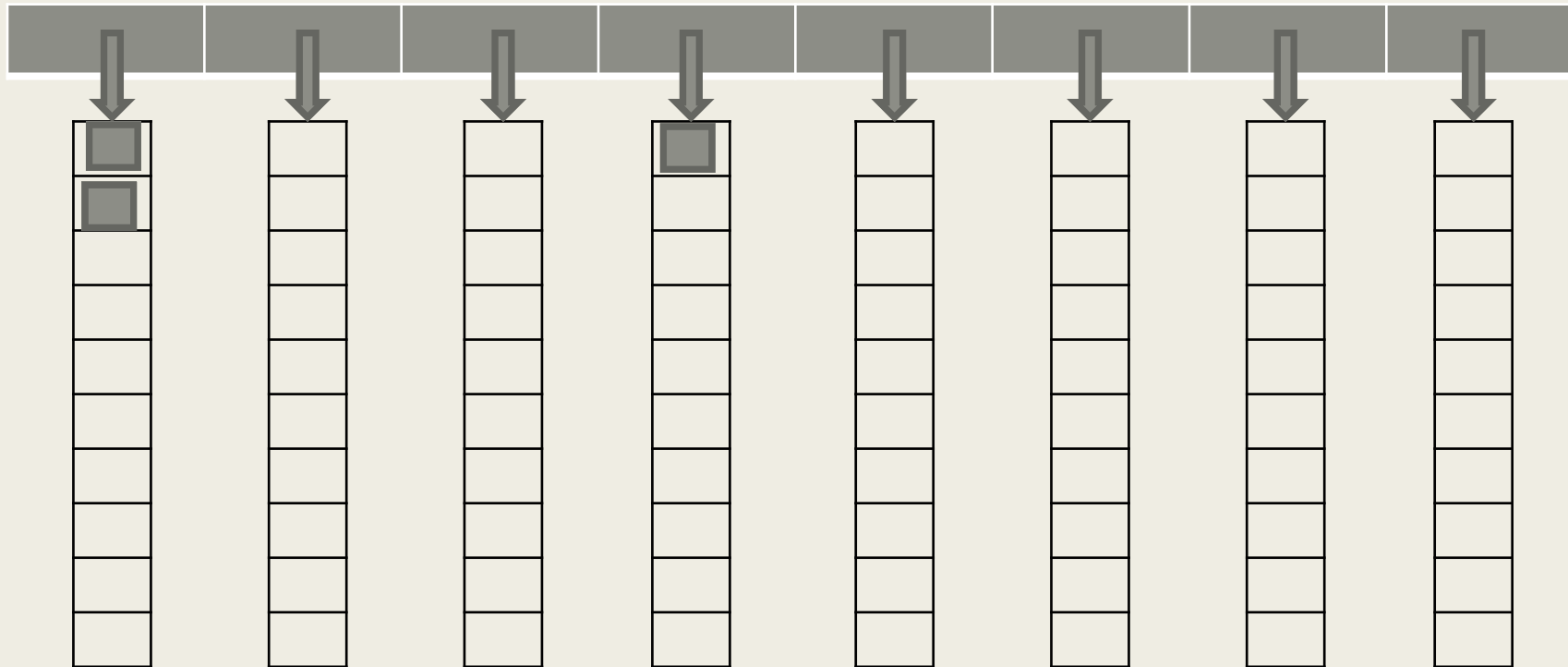
- Each thread could have a shared message “queue” (set of queues)
 - *when one thread wants to “send a message” to another thread, it could enqueue the message in the destination thread’s queue.*
 - *A thread could receive a message by dequeuing the message at the head of its message queue.*



Message-Passing

Shared Queue
Data Structure
Pattern

- Each thread could have a shared message “queue” (set of queues)
 - *any thread can be ‘sending’ a message to any other thread at any time*
 - *any thread can be ‘receiving’ a message at any time*
 - *we need to worry about race conditions on the queues*
 - will have to ‘lock’ a queue when enqueue and dequeue a message
 - **provide mutual exclusion** (*lower level coordination pattern*)



shared
array of pointers
to queues

one queue
per thread

Message-Passing “test” pseudocode

- Each thread is executing this code (SPMD pattern)
 - *each thread is producing and consuming an equal number of messages*

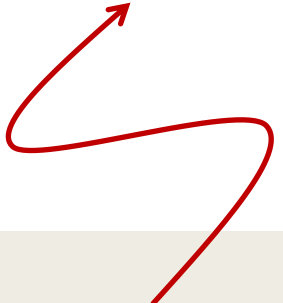
```
for (sent_msgs = 0; sent_msgs < send_max; sent_msgs++) {  
    Send_msg();  
    Try_receive();  
}  
  
while (!Done())  
    Try_receive();
```

This is in main() of omp_msggps.c, omp_msglkc.c

omp_msglkc.c : uses one form of mutual exclusion on the queues: locks omp_msggps.c: uses another form of mutual exclusion on the queues : critical sections

Termination Detection [Done()]

```
queue_size = enqueued - dequeued;  
if (queue_size == 0 && done_sending == thread_count)  
    return TRUE;  
else  
    return FALSE;
```



each thread increments this after
completing its for loop

Sending Messages

omp_msgps.c

```
    msg = random();  
    dest = random() % thread_count;  
#    pragma omp critical  
    Enqueue(queue, dest, my_rank, msg);
```

Receiving Messages

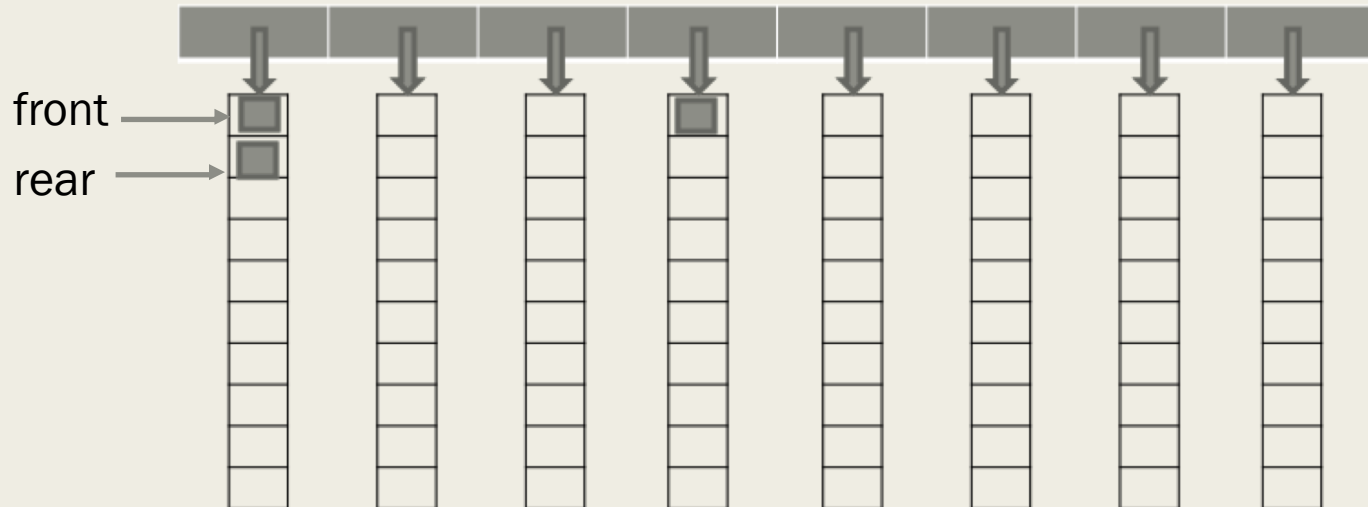
omp_msgps.c

```
    if (queue_size == 0) return;  
    else if (queue_size == 1)  
#        pragma omp critical  
        Dequeue(queue, &src, &msg);  
    else  
        Dequeue(queue, &src, &msg);  
    Print_message(src, msg);
```

Compare these operations

enqueue, or add

```
mesg = random();  
dest = random() % thread_count;  
# pragma omp critical  
  Enqueue(queue, dest, my_rank, mesg);
```



dequeue, or remove

```
if (queue_size == 0) return;  
else if (queue_size == 1)  
#   pragma omp critical  
    Dequeue(queue, &src, &mesg);  
else  
    Dequeue(queue, &src, &mesg);  
  Print_message(src, mesg);
```

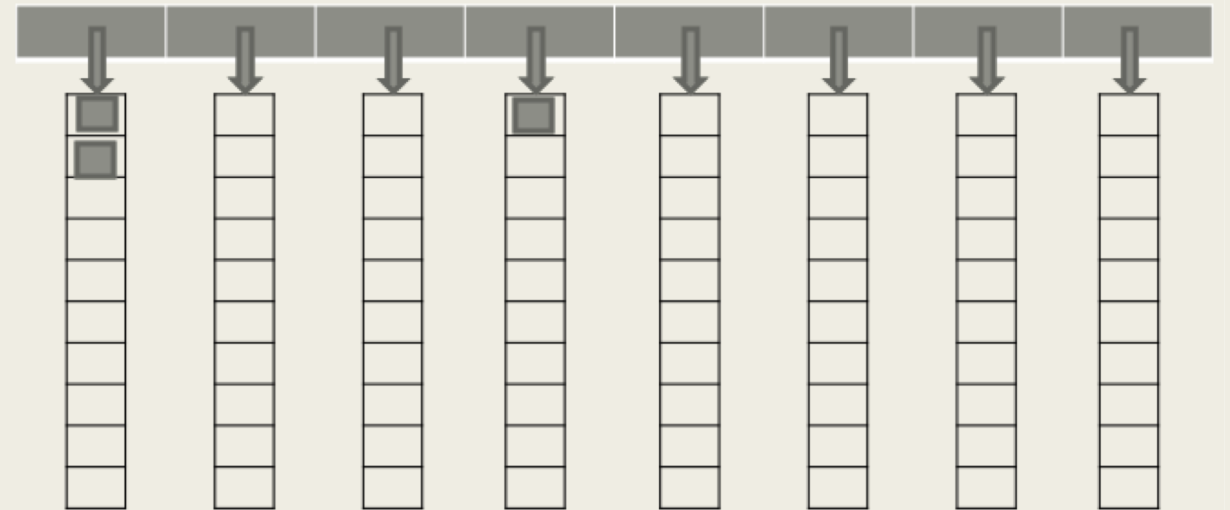
Only one thread ever dequeues a message from its designated queue

As long as there are more than 2 messages on the queue, a call to Dequeue won't conflict with a call to Enqueue

Startup (1)

- When the program begins execution, a single thread, the master thread, will:
 - *get command line arguments and*
 - *allocate an array of pointers to message queues: one for each thread.*
- This array needs to be shared among the threads, since any thread can send to any other thread, and hence any thread can enqueue a message in any of the queues.

Shared Queue
Data Structure
Pattern

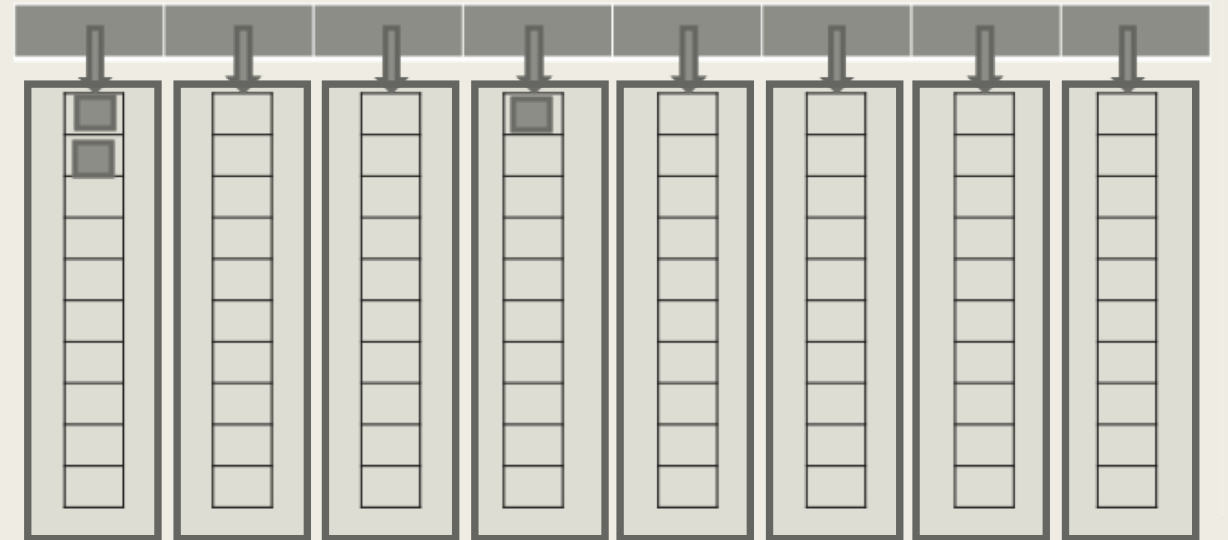


Startup (2): each thread creates its queue in parallel

- One or more threads may finish allocating their queues before some other threads.
- We need an explicit barrier so that when a thread encounters the barrier, it blocks until all the threads in the team have reached the barrier.
- After all the threads have reached the barrier all the threads in the team can proceed.

```
# pragma omp barrier
```

Barrier Coordination
Pattern



Example 'test' program: all threads sending messages concurrently

```
for (msg_number = 0; msg_number < send_max; msg_number++) {  
    Send_msg(msg_queues, my_rank, thread_count, msg_number);  
    Try_receive(msg_queues[my_rank], my_rank);  
}
```

```
# pragma omp atomic
```

Mutual Exclusion
Pattern

```
done_sending++;
```

```
# ifdef DEBUG
```

```
printf("Thread %d > done sending\n", my_rank);
```

```
# endif
```

```
queue_size = enqueued - dequeued;  
if (queue_size == 0 && done_sending == thread_count)  
    return TRUE;  
else  
    return FALSE;
```

```
while (!Done(msg_queues[my_rank], done_sending, thread_count))
```

```
    Try_receive(msg_queues[my_rank], my_rank);
```

omp_msglk.c : uses one form of [mutual exclusion](#) on the queues: locks
omp_msgps.c: uses another form of [mutual exclusion](#) on the queues : critical sections

The Atomic Directive (1)

- Unlike the critical directive, it can only protect critical sections that consist of a single C assignment statement.

```
# pragma omp atomic
```

- Further, the statement must have one of the following forms:

```
x <op>= <expression>;  
x++;  
++x;  
x--;  
--x;
```

The Atomic Directive (2)

- Here <op> can be one of the binary operators

`+, *, -, /, &, ^, |, <<, or >>`

- Many processors provide a special load-modify-store instruction.
- A critical section that only does a load-modify-store can be protected much more efficiently by using this special instruction rather than the constructs that are used to protect more general critical sections.

Example 'test' program: all threads sending messages concurrently

```
for (msg_number = 0; msg_number < send_max; msg_number++) {  
    Send_msg(msg_queues, my_rank, thread_count, msg_number);  
    Try_receive(msg_queues[my_rank], my_rank);  
}
```

```
# pragma omp atomic
```

```
done_sending++;
```

```
# ifdef DEBUG
```

```
printf("Thread %d > done sending\n", my_rank);
```

```
# endif
```



```
queue_size = enqueued - dequeued;  
if (queue_size == 0 && done_sending == thread_count)  
    return TRUE;  
else  
    return FALSE;
```

```
while (!Done(msg_queues[my_rank], done_sending, thread_count))
```

```
    Try_receive(msg_queues[my_rank], my_rank);
```

sending messages using critical

```
void Send_msg(struct queue_s* msg_queues[], int my_rank,  
    int thread_count, int msg_number) {  
    // int mesg = random() % MAX_MSG;  
    int mesg = -msg_number;  
    int dest = random() % thread_count;  
    # pragma omp critical  
        Enqueue(msg_queues[dest], my_rank, mesg);  
    # ifdef DEBUG  
        printf("Thread %d > sent %d to %d\n", my_rank, mesg, dest);  
    # endif  
} /* Send_msg */
```

Critical Sections

- OpenMP provides the option of adding a name to a critical directive:

```
# pragma omp critical(name)
```

- When we do this, two blocks protected with critical directives with different names can be executed simultaneously.
- However, the names are set during compilation, and we want a different critical section for each thread's queue.

Locks

- A lock consists of a data structure and functions that allow the programmer to explicitly enforce mutual exclusion in a critical section.
- OpenMP provides a *functional* mechanism for this as an alternative to the pragma construct



Locks

```
/* Executed by one thread */  
Initialize the lock data structure;  
.  
.  
.  
/* Executed by multiple threads */  
Attempt to lock or set the lock data structure;  
Critical section;  
Unlock or unset the lock data structure;  
.  
.  
.  
/* Executed by one thread */  
Destroy the lock data structure;
```

Simple example

```
#include <omp.h>

lock_t myLock;

int main(...) {
    (void) omp_init_lock(&myLock);
    #pragma omp parallel
    {
        (void) TheCookieJar();
    } // End of parallel region
    (void) omp_destroy_lock(&myLock); }
```

```
void TheCookieJar() {
    (void) omp_set_lock(&myLock); // acquire lock
    (void) grabMyFavouriteCookie(); // get cookie
    (void) omp_unset_lock(&myLock); // release lock

}
```


Using Locks in the Message-Passing Program

```
# pragma omp critical  
    /* q_p = msg_queues[dest] */  
    Enqueue(q_p, my_rank, msg);
```

```
/* q_p = msg_queues[dest] */  
omp_set_lock(&q_p->lock);  
Enqueue(q_p, my_rank, msg);  
omp_unset_lock(&q_p->lock);
```

Using Locks in the Message-Passing Program

```
# pragma omp critical  
/* q_p = msg_queues[my_rank] */  
Dequeue(q_p, &src, &msg);
```

```
/* q_p = msg_queues[my_rank] */  
omp_set_lock(&q_p->lock);  
Dequeue(q_p, &src, &msg);  
omp_unset_lock(&q_p->lock);
```

Some Caveats

1. You shouldn't mix the different types of mutual exclusion for a single critical section.
2. There is no guarantee of fairness in mutual exclusion constructs.
3. It can be dangerous to “nest” mutual exclusion constructs.

Locks vs. critical

- critical focuses on the code section

```
#   pragma omp critical
    /* q_p = msg_queues[my_rank] */
    Dequeue(q_p, &src, &msg);
```

- locks are about the data

```
/* q_p = msg_queues[my_rank] */
omp_set_lock(&q_p->lock);
Dequeue(q_p, &src, &msg);
omp_unset_lock(&q_p->lock);
```

A real implementation would place the locking inside the Dequeue function:
has potential of being more fine-grained (smaller amount of code is locked)

Locks are subject to more errors

- forgetting to initialize lock variable
- deadlock
- unsetting a lock from another thread

You try the code

- Work with a neighbor
- `shared/activities/omp_msg`
 - *original code:* `examples/Pacheco-ipp-source/ch5/omp_msg`
- Note the two different implementations, which have different queue code files:
 - *queue.c and queue.h*
 - *queue_lk.c and queue_lk.h*

Example 'test' using *critical* & *atomic*

uses queue.c/h

```
/* File:  omp_msggps.c
*
* Purpose: Simulate message-passing using OpenMP. Uses critical and
*          atomic directives to protect critical sections.
*
* Compile: gcc -g -Wall -fopenmp
*          -o omp_msggps omp_msggps.c queue.c
*          needs queue.h
* Usage:  ./omp_msggps <number of threads> <number of messages each thread sends>
*
* Input:  None
* Output: Source, destination and contents of each message received.
*
* Notes:
* 1. DEBUG flag for more verbose output
*/
```

Example 'test' using *OpenMP* locks & *atomic*

```
/* File:  omp_msglk.c                                uses queue_lk.c/h
 *
 * Purpose: Simulate message-passing using OpenMP. This version uses
 *           an atomic directive and OpenMP locks to protect critical
 *           sections.
 *
 * Compile: gcc -g -Wall -fopenmp -o omp_msglk omp_msglk.c queue_lk.c
 *           needs queue_lk.h
 * Usage:  ./omp_msglk <number of threads> <number of messages each thread sends>
 *
 * Input:   None
 * Output:  Source, destination and contents of each message received.
 *
 * Notes:
 * 1. DEBUG flag for more verbose output
 * 2. This version uses locks to control access to the message queues.
 */
```


Concluding Remarks (1)

- OpenMP is a standard for programming shared-memory systems.
- OpenMP uses both special functions and preprocessor directives called pragmas.
- OpenMP programs start multiple threads rather than multiple processes.
- Many OpenMP directives can be modified by clauses.

Concluding Remarks (2)

- A major problem in the development of shared memory programs is the possibility of race conditions.
- OpenMP provides several mechanisms for insuring mutual exclusion in critical sections.
 - *Critical directives*
 - *Named critical directives*
 - *Atomic directives*
 - *Simple locks*

Concluding Remarks (3)

- By default most systems use a block-partitioning of the iterations in a parallelized for loop.
- OpenMP offers a variety of scheduling options.
- In OpenMP the scope of a variable is the collection of threads to which the variable is accessible.

Concluding Remarks (4)

- A reduction is a computation that repeatedly applies the same reduction operator to a sequence of operands in order to get a single result.